

Bases de Datos II

Tarea Corta 2

Equipo de trabajo:

Granados Retana, Diego - 2022158363 Granados Retana, Daniel - 2022104692 Mora Montes, Diego - 2022104866 Gutierrez Conejo, Eduardo - 2019073558 Cardona Quesada, Jose Ricardo - 2021022613

Manual de instalación

A continuacion se presenta un manual de instalacion para distintos motores de bases de datos junto con respectivos codigos para implementar backups y restaurar datos a través de una cuenta AWS.

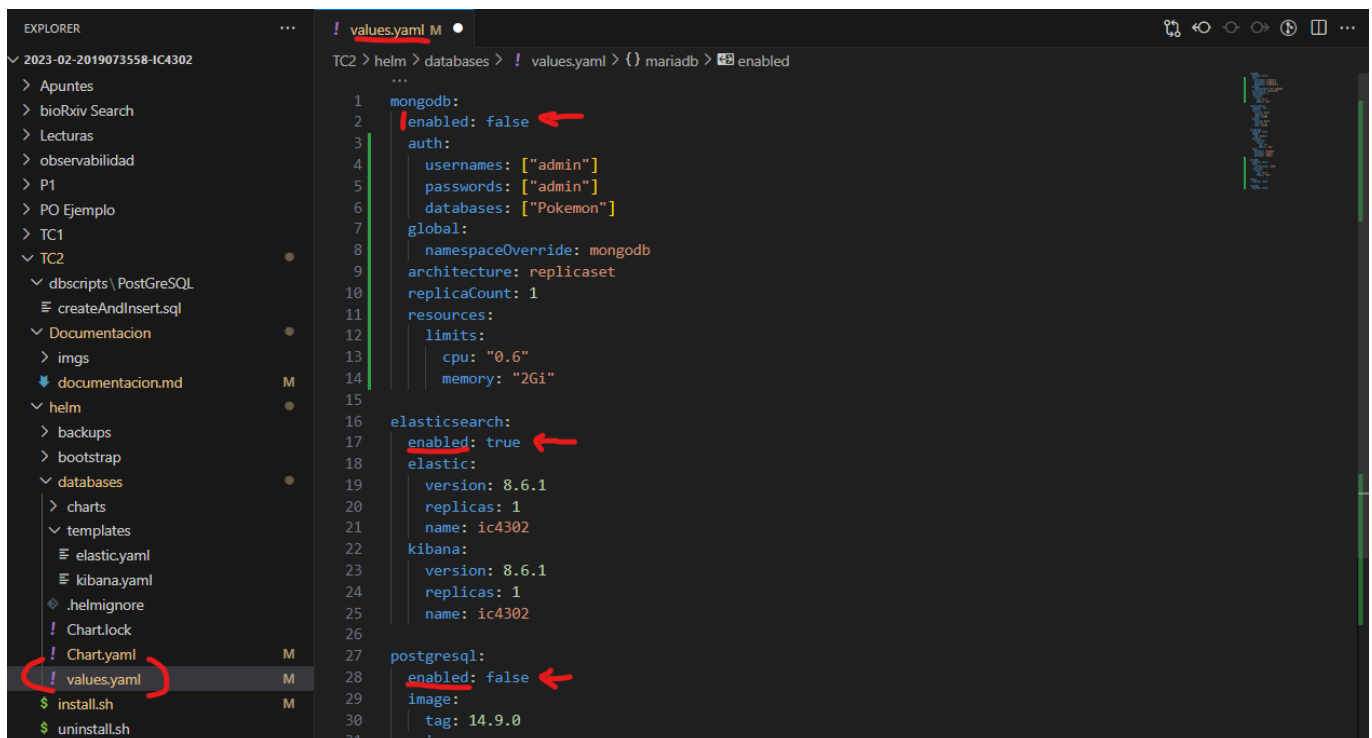
Paso 1. Selección del Motor de Bases de datos a probar

Dentro de la carpeta para la TC2 se incluyen varios motores de bases de datos, la lista de todos estos es:

• MariaDB • PostgreSQL • Elasticsearch • MongoDB • Neo4J • CouchDB

Una vez se haya seleccionado la base de datos para la que se desea probar backups y restauración se hace lo siguiente:

Se ingresa al path `TC2\helm\databases\values.yaml` donde se va a cambiar ciertos parámetros para hacer backups solo en la base deseada e iniciar solo la base deseada. Por cada base de datos saldrá el siguiente parámetro:



```
EXPLORER
2023-02-2019073558-IC4302
  > Apuntes
  > bioRxiv Search
  > Lecturas
  > observabilidad
  > P1
  > PO Ejemplo
  > TC1
  > TC2
    > dbscripts\ PostgreSQL
    > createAndInsert.sql
    > Documentacion
    > imgs
    > documentacion.md
    > helm
      > backups
      > bootstrap
      > databases
        > charts
        > templates
        > elasticsearch.yaml
        > kibana.yaml
        > .helmignore
        > Chart.lock
        > Chart.yaml
        > values.yaml
      > install.sh
      > uninstall.sh

! values.yaml M
TC2 > helm > databases > ! values.yaml > {} mariadb > enabled
...
1  mongodb:
2  (enabled: false
3  auth:
4    usernames: ["admin"]
5    passwords: ["admin"]
6    databases: ["Pokemon"]
7  global:
8    namespaceOverride: mongodb
9  architecture: replicaset
10 replicaCount: 1
11 resources:
12   limits:
13     cpu: "0.6"
14     memory: "2Gi"
15
16 elasticsearch:
17   enabled: true
18   elastic:
19     version: 8.6.1
20     replicas: 1
21     name: ic4302
22   kibana:
23     version: 8.6.1
24     replicas: 1
25     name: ic4302
26
27 postgresql:
28   enabled: false
29   image:
30     tag: 14.9.0
31   primary:
```

Enabled hace referencia a si se quiere inicializar o no ese motor de base de datos, la recomendación es no correr más de uno al mismo tiempo.

En caso de querer correr ese motor cambiar **Enabled: True** por la versión actual, y al resto de motores ingresar **Enabled: False** para apagarlos

Una vez hecho esto se debe ingresar a **TC2\helm\backups\values.yaml**, y hacer lo mismo con las bases de datos que aparecen.

IMPORTANTE: En caso de querer ejecutar backups a través de elastic **NO es necesario** ingresar a **TC2\helm\backups\values.yaml** a habilitar la base también

Aqui nose si falta algo para habilitar si se quieren backups o restores.

Finalmente, antes de ejecutar el código

Paso 2. Ejecución del Código

Una vez se haya escogido la base, el siguiente paso es ejecutar los comandos para levantarla con sus respectivos parámetros, para esto ingresamos a cualquier terminal desde la cuál se puedan ejecutar comandos Linux. Una vez aqui abriremos la carpeta desde la terminal:

```
root@RENE-LAPTOP-G7-7588:/mnt/d/OneDrive/Desktop/PrograJose/2023-02-2019073558-IC4302# cd TC2
root@RENE-LAPTOP-G7-7588:/mnt/d/OneDrive/Desktop/PrograJose/2023-02-2019073558-IC4302/TC2# cd helm
root@RENE-LAPTOP-G7-7588:/mnt/d/OneDrive/Desktop/PrograJose/2023-02-2019073558-IC4302/TC2/helm#
```

Una vez en la carpeta donde hayamos descargado el proyecto se insertarán los siguientes comandos:

```
cd TC2 cd helm
```

Una vez aqui la terminal se encuentra en la carpeta donde se pueden ejecutar los archivos para la instalacion y desinstalación de las bases de datos.

Para Instalar la base deseada con los parámetros especificados

En la terminal primero se recomienda ejecutar:

```
dos2unix install.sh
```

Seguido de esto ejecutamos el comando: **./install.sh**

Una vez hecho esto la terminal comenzará a instalar todos los deployments necesarios para la ejecución correcta del programa:

```

root@RENE-LAPTOP-G7-7588:/mnt/d/OneDrive/Desktop/PrograJose/2023-02-2019073558-IC4302/TC2/helm# dos2unix install.sh
dos2unix: converting file install.sh to Unix format...
root@RENE-LAPTOP-G7-7588:/mnt/d/OneDrive/Desktop/PrograJose/2023-02-2019073558-IC4302/TC2/helm# ./install.sh
2023/10/20 11:09:43.130990 system_key.go:129: cannot determine nfs usage in generateSystemKey: cannot parse mountinfo: incorrect number of tail fields, expected 3 but found 4
2023/10/20 11:09:43.135607 cmd_run.go:1046: WARNING: cannot create user data directory: cannot determine SELinux status: failed to obtain SELinux mount path: incorrect number
of tail fields, expected 3 but found 4
Saving 1 charts
Downloading eck-operator from repo https://helm.elastic.co
Deleting outdated charts
2023/10/20 11:09:45.428497 system_key.go:129: cannot determine nfs usage in generateSystemKey: cannot parse mountinfo: incorrect number of tail fields, expected 3 but found 4
2023/10/20 11:09:45.433184 cmd_run.go:1046: WARNING: cannot create user data directory: cannot determine SELinux status: failed to obtain SELinux mount path: incorrect number
of tail fields, expected 3 but found 4
Release "bootstrap" does not exist. Installing it now.
NAME: bootstrap
LAST DEPLOYED: Fri Oct 20 11:09:45 2023
NAMESPACE: default
STATUS: deployed
REVISION: 1
TEST SUITE: None

```

Una vez este código se verá lo siguiente:

```

Release "databases" does not exist. Installing it now.
NAME: databases
LAST DEPLOYED: Fri Oct 20 11:30:02 2023
NAMESPACE: default
STATUS: deployed
REVISION: 1
TEST SUITE: None
2023/10/20 11:31:02.794566 system_key.go:129: cannot determine nfs usage in generateSystemKey: cannot parse mountinfo: incorrect number of tail fields, expected 3 but found 4
2023/10/20 11:31:02.802229 cmd_run.go:1046: WARNING: cannot create user data directory: cannot determine SELinux status: failed to obtain SELinux mount path: incorrect number of tail f
lds, expected 3 but found 4
2023/10/20 11:31:03.243505 system_key.go:129: cannot determine nfs usage in generateSystemKey: cannot parse mountinfo: incorrect number of tail fields, expected 3 but found 4
2023/10/20 11:31:03.248048 cmd_run.go:1046: WARNING: cannot create user data directory: cannot determine SELinux status: failed to obtain SELinux mount path: incorrect number of tail f
lds, expected 3 but found 4
Release "backups" does not exist. Installing it now.
NAME: backups
LAST DEPLOYED: Fri Oct 20 11:31:03 2023
NAMESPACE: default
STATUS: deployed
REVISION: 1
TEST SUITE: None
root@RENE-LAPTOP-G7-7588:/mnt/d/OneDrive/Desktop/PrograJose/2023-02-2019073558-IC4302/TC2/helm#

```

Este código habilita la base deseada para las pruebas que se quieran ejecutar con ella. Cada una de las bases es distinta.

Ejecución de Bases y pruebas

IMPORTANTE

Para poder verificar si los archivos correspondientes a una de las bases se encuentran en el bucket, se puede hacer lo siguiente:

En la terminal, con el proyecto abierto, nos aseguramos de volver a la carpeta TC2 y en caso de estar adentro de Helm, hay que regresar. Para esto, se ejecuta el comando `cd ..` hasta regresar a la dirección del TC2.

Una vez en esta dirección, se ejecutan los comandos:

```
cd utils
```

```
dos2unix install.sh
```

```
./install.sh
```

Este código va a instalar un pod en la red de kubernetes donde se pueden ejecutar comandos para verificar si el bucket tiene archivos en la dirección que se quiere revisar. Al finalizar se podrá ver lo siguiente:

```

root@RENE-LAPTOP-G7-7588:/mnt/d/OneDrive/Desktop/PrograJose/2023-02-2019073558-IC4302/tc2/Utils# ./install.sh
Using default tag: latest
latest: Pulling from aws-cli/aws-cli
daa81221dec1: Already exists
ac9dcdbde20ae: Pull complete
0e4f469ee5ea: Pull complete
0f0d26aefdf3: Pull complete
382122d245f9: Pull complete
Digest: sha256:e668c96d1df44d3796e4faf00121e9f38fdfed8c9b1439eff0aa6a473c694417
Status: Downloaded newer image for public.ecr.aws/aws-cli/aws-cli:latest
public.ecr.aws/aws-cli/aws-cli:latest
pod/debug-pod created
bash-4.2#

```

Aquí se pueden ingresar comandos para ver las distintas direcciones donde están almacenados los backups, normalmente estas direcciones siguen el patrón:

2019073558 / [nombre base]

EJ: **2019073558/elastic 2019073558/postgresql**

El comando para ver los archivos en esa dirección es:

aws s3 ls s3://tec-ic4302-02-2023/2019073558/[nombre Base]/

EJ:

aws s3 ls s3://tec-ic4302-02-2023/2019073558/postgresql/

aws s3 ls s3://tec-ic4302-02-2023/2019073558/elastic/

Al ejecutar estos comandos se puede ver lo siguiente:

```

bash-4.2# aws s3 ls s3://tec-ic4302-02-2023/2019073558/elastic/
                PRE indices/
                PRE tests-52XAchX2SnePp29GmISIRw/
2023-10-20 23:31:51      607 index-0
2023-10-20 23:31:51        8 index.latest
2023-10-20 23:31:50     212 meta-ijst37f1RPeqwsVBQeJ1ew.dat
2023-10-20 23:31:50     320 snap-ijst37f1RPeqwsVBQeJ1ew.dat
bash-4.2# aws s3 ls s3://tec-ic4302-02-2023/2019073558/postgresql/
2023-10-16 01:37:12        0 Pokemon-202310160136.sql
2023-10-16 01:40:59        0 Pokemon-202310160140.sql
2023-10-16 03:12:57        0 Pokemon-202310160311.sql
2023-10-16 04:02:05     3087 Pokemon-202310160400.sql
2023-10-17 20:34:14     3087 Pokemon-202310172031.sql
2023-10-18 14:56:12     3087 Pokemon-202310181455.sql
bash-4.2#

```

Claramente se despliegan los distintos backups que han sido creados hasta el momento para las respectivas bases.

Elastic Search con Kibana

A continuación se muestra una guía para hacer backups y restores en la base elasticsearch a través de su interfaz kibana. Para este paso se asume que ya se habilitó la base y se ejecutó el script de instalación desde la terminal.

Lo primero que se debe hacer es tener instalada una aplicación para poder visualizar el kubernetes deployment y fácilmente obtener la información importante de los pods.

Para esta funcionalidad se recomienda hacer uso de **LENS**, el resto de esta guía se basará en el uso de esta aplicación para conectar los pods.

Desde Lens lo primero que haremos será entrar el deployment de docker desktop y nos iremos a la sección de **PODS**, una vez aquí entraremos al siguiente pod:

The screenshot shows the Kubernetes Lens application interface. On the left, the 'Pods' tab is selected, displaying a table with 3 items. The table has columns for Name, Namespace, Container, CPU, and Memory. The pod 'ic4302-kb-5f796466db-vcnrt' is highlighted. On the right, the details for this pod are shown. The 'Containers' section lists 'kibana' as the only container. The 'Ports' section shows a port mapping for 'https: 5601/TCP'. A red arrow points to this link. The 'Status' is 'running, ready' and the 'Image' is 'docker.elastic.co/kibana/kibana:8.6.1'.

Name	Namespace	Container	CPU	Memory
elastic-operator	default		N/A	N/A
ic4302-es-defal	default		N/A	N/A
ic4302-kb-5f796466db-vcnrt	default		N/A	N/A

Pod: ic4302-kb-5f796466db-vcnrt

```
"${init_config_initialized_flag}" ]]; then echo "Kibana configuration already initialized." exit 0 fi echo "Setup Kibana configuration" ln -sf /mnt/elastic-internal/kibana-config/* /mnt/elastic-internal/kibana-config-local/ touch "${init_config_initialized_flag}" echo "Kibana configuration successfully prepared."
```

Containers

- kibana

CPU Memory Filesystem

Metrics not available at the moment

Status: running, ready

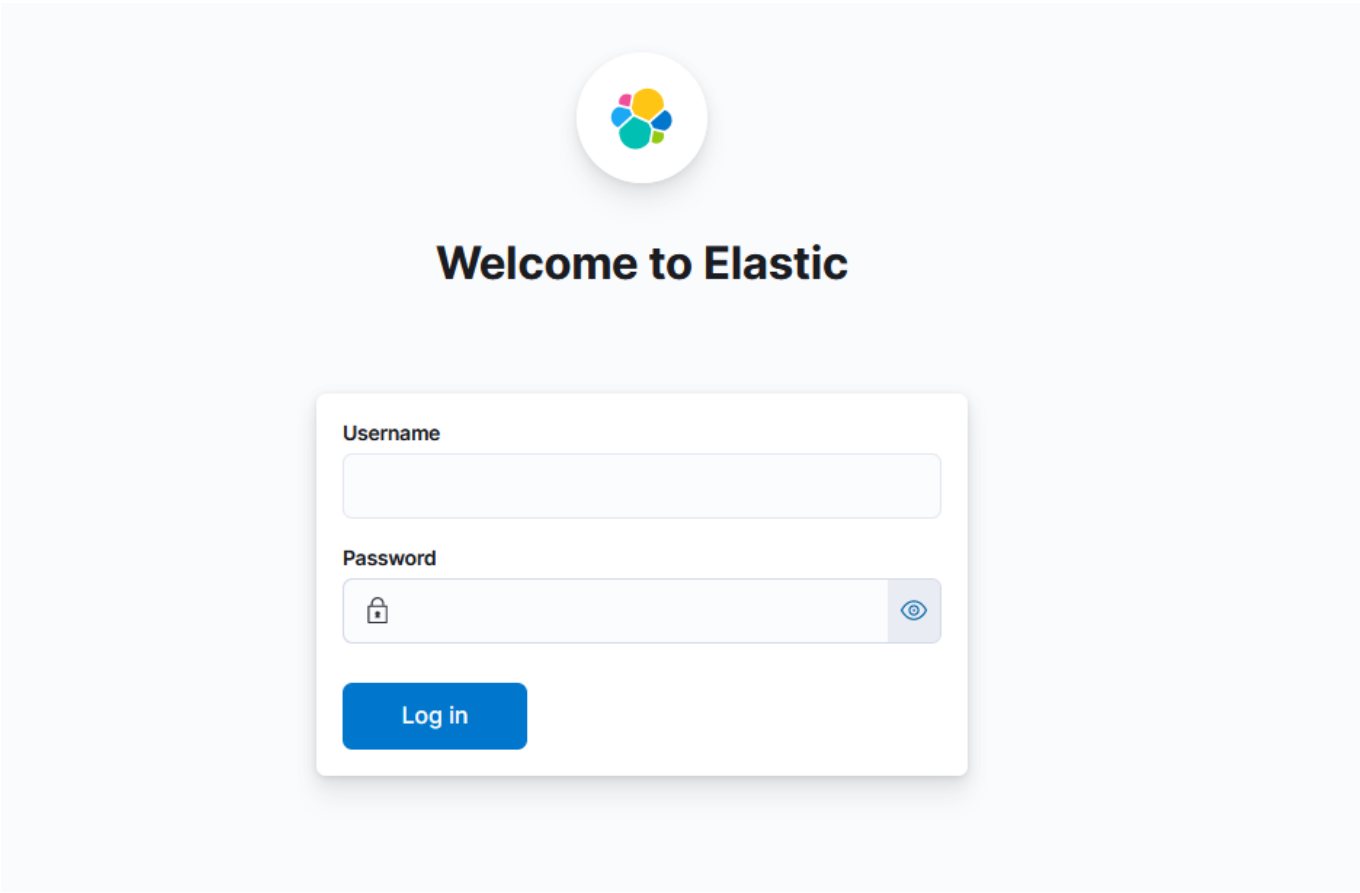
Image: docker.elastic.co/kibana/kibana:8.6.1

Ports: https: 5601/TCP Stop/Remove

Environment: —

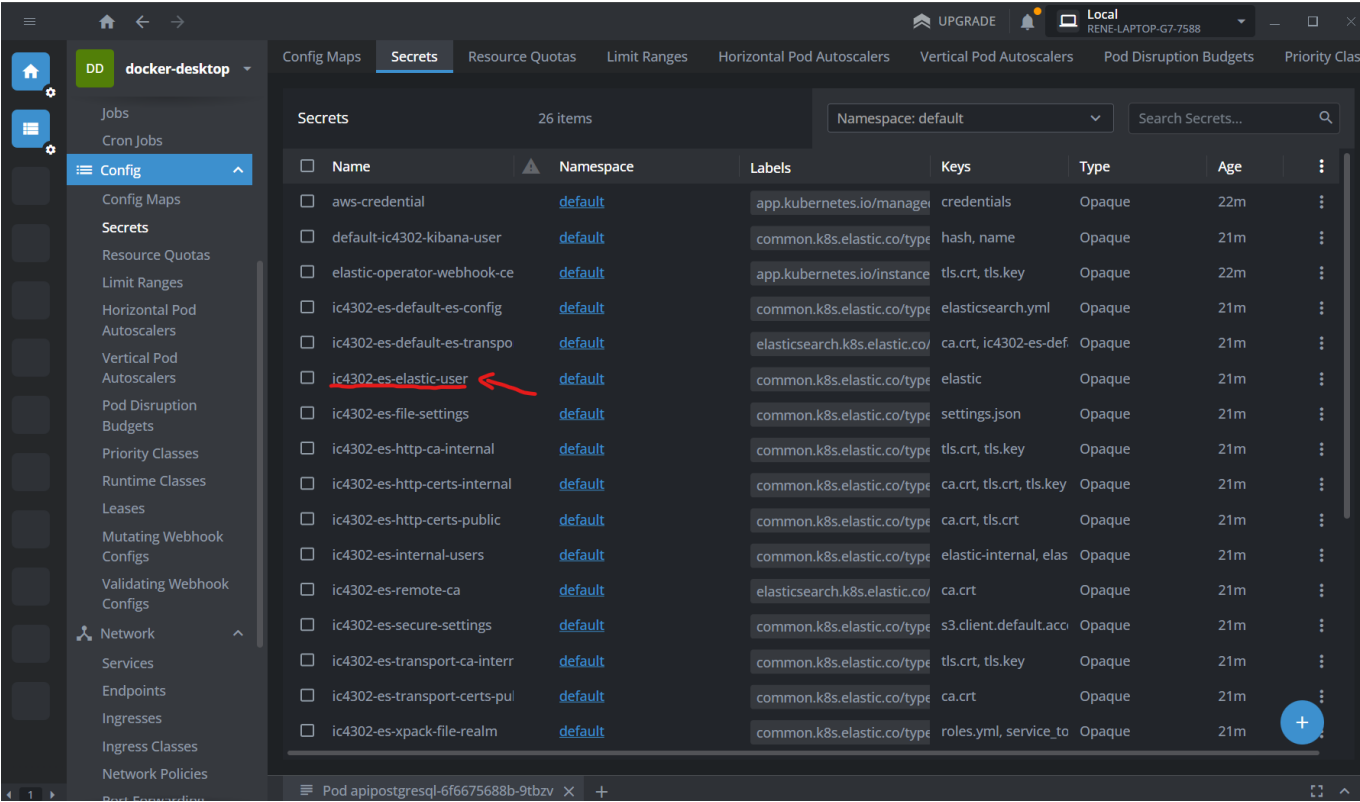
Al abrir el pod se debe buscar la sección señalada a la derecha, esta contiene un link que nos permitirá acceder de forma local a la interfaz Kibana para interactuar con la base de datos elasticsearch.

Una vez se haya hecho forward del port y se haya ingresado al sitio aparecerá lo siguiente:

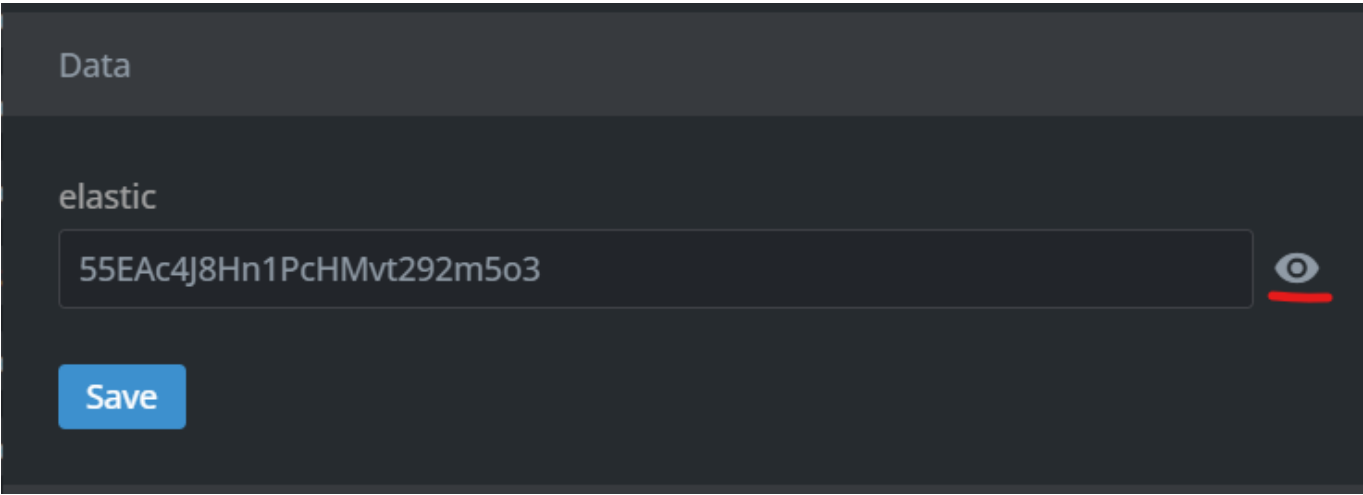


Se solicita un usuario y una contraseña para poder ingresar al servicio Kibana, para obtener estos datos volvemos a **LENS** y buscamos la sección:

Config/Secrets, una vez aqui abrimos el siguiente secret:

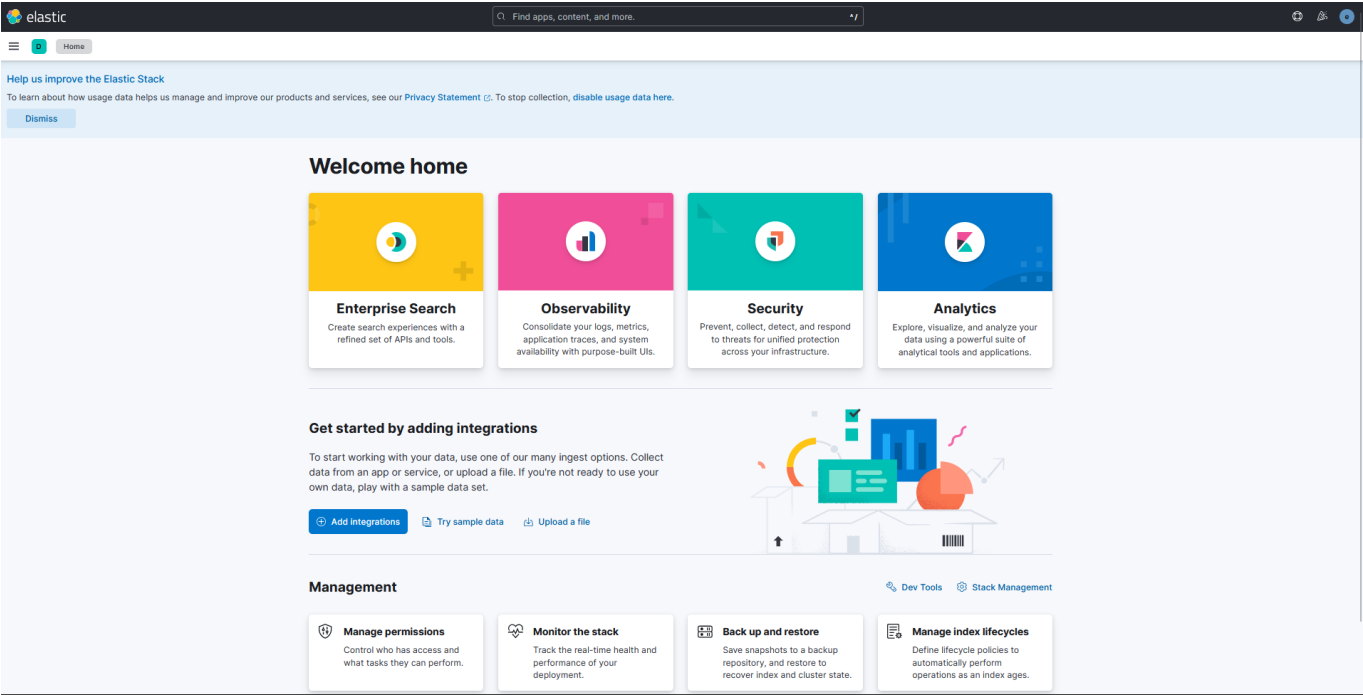


Una vez aqui buscamos los siguientes datos:

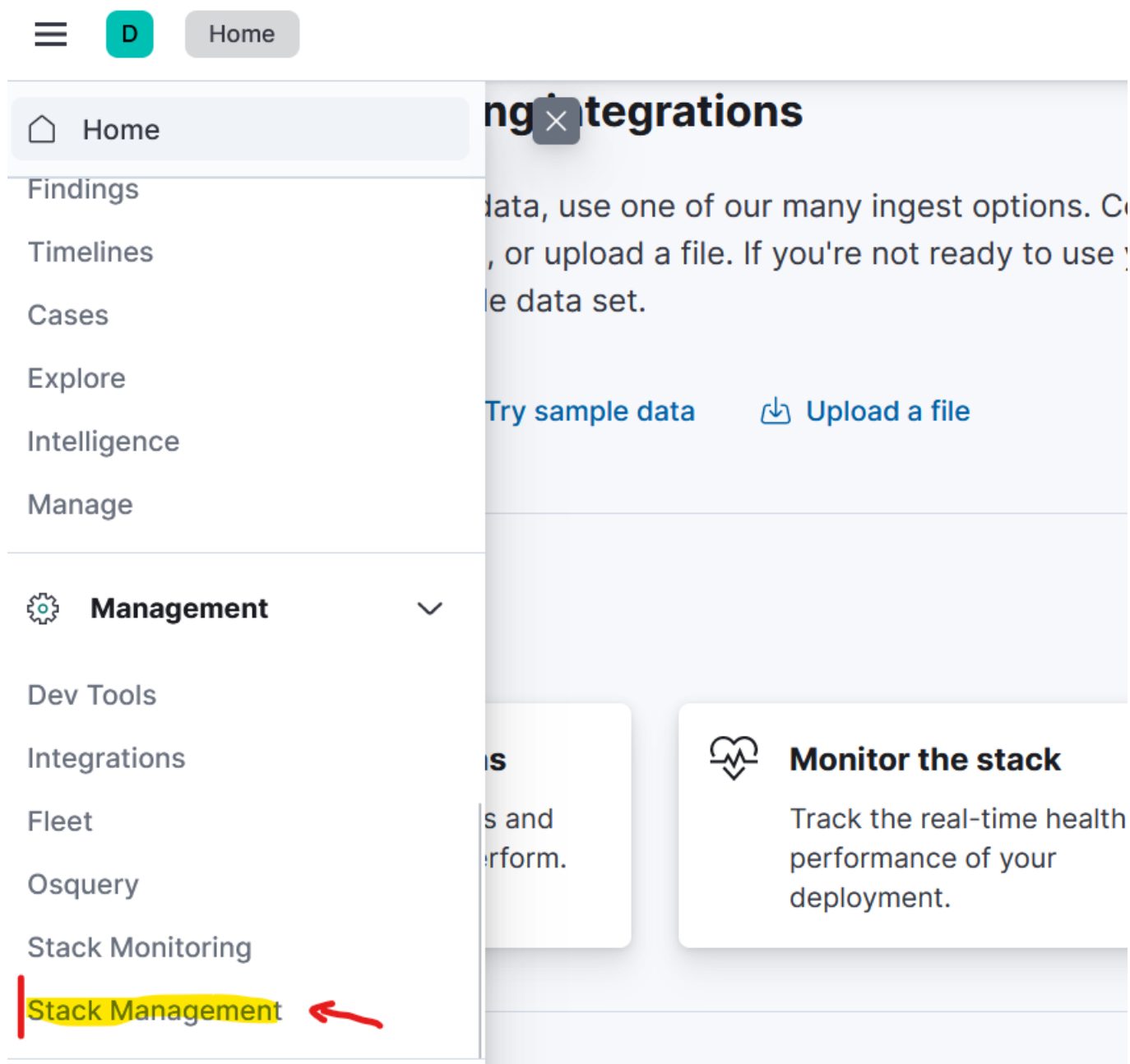


Estos corresponden a mi usuario y contraseña de elastic search. Por lo tanto copio estas credenciales y las pongo en la interfaz. El usuario será elastic y la contraseña el otro valor.

Después de ingresar las credenciales correctamente saldra una imagen similar a esta:

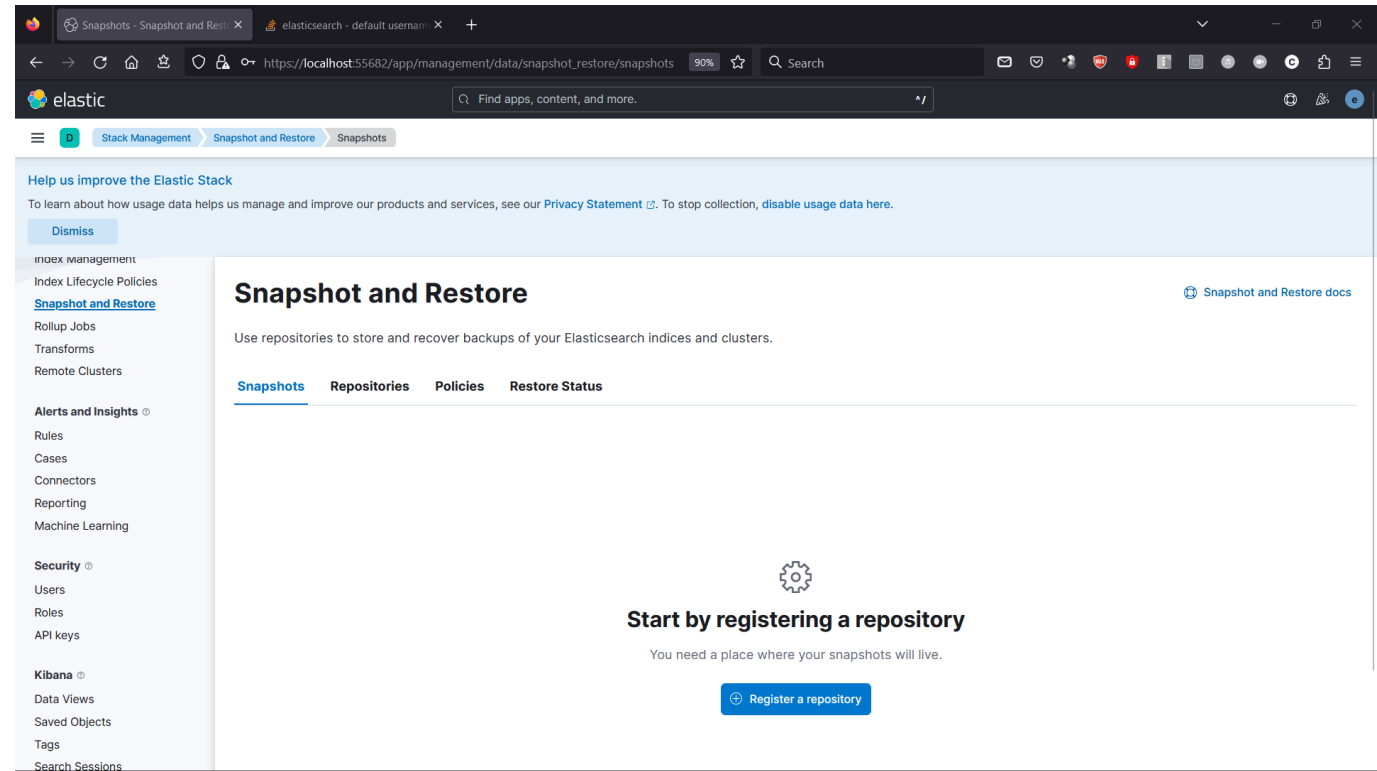


En esta pantalla buscaremos la opcion stack management e ingresaremos



Una vez en esta interfaz, se busca la sección con el nombre **Snapshot and Restore**, en elastic los backups son llamados de esta forma.

Para restaurar o guardar backups lo primero es definir la dirección donde se van a guardar/obtener. Para esto se va a agregar el repositorio donde vamos a manejar nuestros backups de elastic, este hace referencia a la dirección del bucket de AWS y el cliente que vamos a usar para guardar los datos correspondientes a los snapshots.



Una vez aqui se selecciona **Register a repository** para crearlo. Una vez adentro:

Seleccionamos el tipo de repositorio, en este caso debe de ser AWS S3 y se le pone el nombre que quiera al repositorio.

Repository name

A unique name for the repository.

Name

BackupsElastic

Repository name is required.

Repository type

Storage location for your snapshots. [Learn more about repository types.](#)

Azure

Learn more

Select

Google Cloud Storage

Learn more

Select

AWS S3

Learn more

Selected

Shared file system

Learn more

Select

Read-only URL

Learn more

Select

En la siguiente pantalla se solicitan los datos que le queremos poner al repositorio. En esta sección los únicos campos relevantes son:

9 / 35

Register repository

'BackupsElastic' settings

[AWS S3 repository docs](#)**Client**

The name of the AWS S3 client.

Client

default

Bucket

The name of the AWS S3 bucket to use for snapshots.

Bucket (required)

tec-ic4302-02-2023

Base path

The bucket path to the repository data.

Base path

2021022613/elastic

Snapshot compression

Compresses the index mapping and setting files for snapshots. Data files are not compressed.



Compress snapshots

En el campo Client ingresamos: **default**

En el campo bucket ingresamos: **tec-ic4302-02-2023**

En el campo Base path se debe ingresar: **2019073558/elastic**

Es importante anotar este base path para posteriormente usarlo al definir la política para los backups.

El resto de campos para el repositorio se dejan por default y se finaliza la creación de este. El resultado de este proceso se verá similar a esto:

Snapshot and Restore

[Snapshot and Restore docs](#)

Use repositories to store and recover backups of your Elasticsearch indices and clusters.

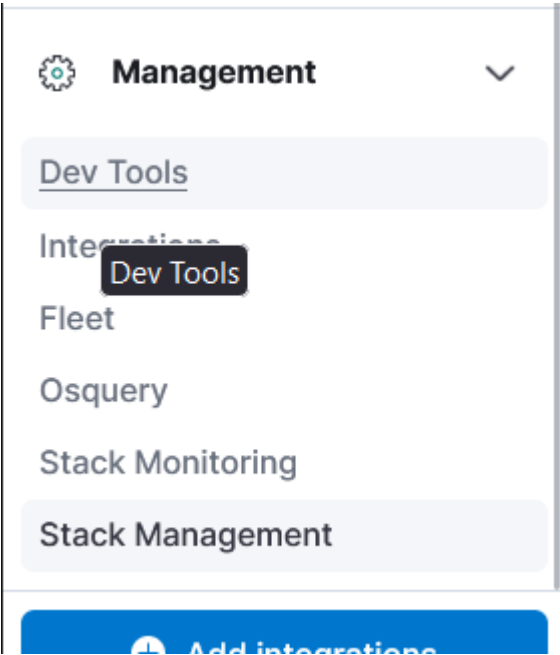
Snapshots **Repositories** Policies Restore Status

Search...		Type ▾	Reload	Register repository
☐ Name ↑	Type	Actions		
☐ BackupsElastic	AWS S3	Edit	Delete	

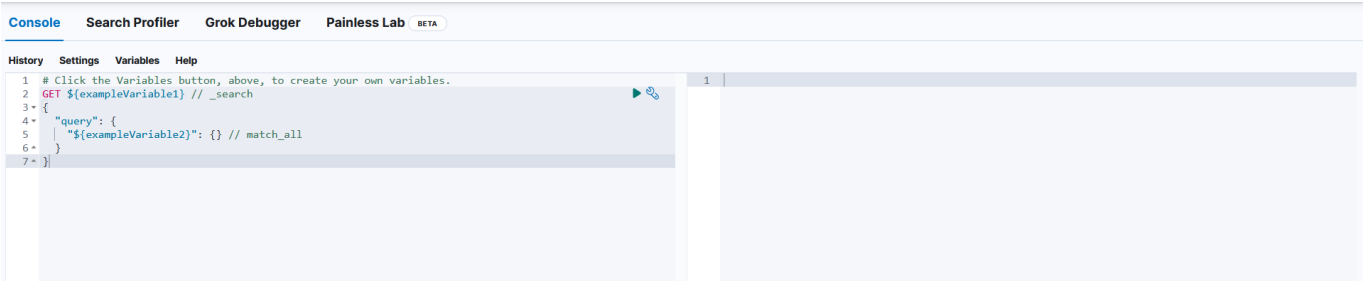
Para hacer pruebas

Cabe aclarar que el repo será adonde guardemos los backups, pero aún no se ha creado ninguna política para crear backups o se han creado indices para los cuáles probar esto. Por lo tanto antes de seguir con el proceso de backups se van a insertar indices en la base elasticsearch manualmente, posteriormente se mostrará como hacer un backup con esta. Finalmente se borrarán los datos y se tratará de restaurarlos usando el snapshot.

Primero, para ingresar los datos a la base es necesario volver al menú principal de la interfaz de Kibana, una vez aqui buscamos la siguiente sección:



Ingresamos a Dev Tools, y se presentará la siguiente interfaz:



Aqui escribimos los siguientes comandos:

PUT /indice-backup

POST /indice-backup/_doc { "titulo": "Prueba Backups", "contenido": "Este documento es una prueba de backups", "integrantes": ["Granados Retana Diego", "Granados Retana Daniel","Mora Montes Diego","Gutierrez Conejo Eduardo","Cardona Quesada Jose Ricardo"] }

GET /indice-backup/_search { "query": { "match": { "titulo": "Prueba Backups" } } }



Una vez hecho esto se insertaron datos y un índice para el cuál podemos hacer backups

Snapshots

Para guardar snapshots de nuestra base completa o un índice en ella hay dos maneras, se puede hacer manualmente a través de comandos o automáticamente definiendo una política de guardado.

Método Manual

Para hacer un backup manualmente, desde la misma interfaz de Dev Tools se puede ingresar un comando como este:

```
PUT /_snapshot/ [NOMBRE_REPOSITORIO] / [NOMBRE_SNAPSHOT] { "indices": "[NOMBRE INDICE]",  
"ignore_unavailable": true, "include_global_state": false }
```

Donde se reemplaza la dirección con el nombre del índice al que queremos tomar un snapshot. En este caso para el ejemplo se usara el siguiente:

```
PUT /_snapshot/BackupsElastic/indice-backup_20231020 { "indices": "indice-backup",  
"ignore_unavailable": true, "include_global_state": false }
```

Para revisar que este snapshot se encuentra disponible en el repositorio y en AWS se puede revisar de dos maneras:

La primera opción es ingresar a la sección de Stack Management y Snapshot-Restore dentro de la interfaz de Kibana. Una vez aqui se verá lo siguiente:

Snapshot and Restore

[Snapshot and Restore docs](#)

Use repositories to store and recover backups of your Elasticsearch indices and clusters.

Snapshots							
Repositories Policies Restore Status							
Search...							
Repository ▼ Reload							
☐	Snapshot	Repository	Indices	Shards	Failed shards	Date created ↓	Duration
☐	indice-backup_20231020	BackupsElastic	1	1	0	Oct 20, 2023 5:33 PM CST	3s
Rows per page: 20 ▼							
< 1 >							

Si se logro crear correctamente el snapshot, va a aparecer en esta sección. La segunda opción es a través de el debug-pod, como se menciona en la sección IMPORTANTE, previo a esta. Al utilizar el debug pod se puede observar el índice recién almacenado

```
bash-4.2# aws s3 ls s3://tec-ic4302-02-2023/2019073558/elastic/  
PRE indices/  
PRE tests-52XAchX2SnePp29GmISIRw/  
2023-10-20 23:31:51 607 index-0  
2023-10-20 23:31:51 8 index.latest  
2023-10-20 23:31:50 212 meta-ijst37f1RPeqwsVBQeJ1ew.dat  
2023-10-20 23:31:50 320 snap-ijst37f1RPeqwsVBQeJ1ew.dat
```

Automático

El segundo método para hacer backups, es estableciendolos automáticamente a través de un policy, usando uno de estos, se pueden establecer la creación de backups cada cierto tiempo automáticamente, junto con un tiempo para el cual cuando se pase sean eliminados automáticamente también.

Para esto en la misma sección de Snapshots and Restore dentro de la Interfaz de Kibana se accede a la sección de Snapshot and Restore y una vez ahí se entra a Políticas/Create a Policy:

Snapshot and Restore

[Snapshot and Restore docs](#)

Use repositories to store and recover backups of your Elasticsearch indices and clusters.

[Snapshots](#) [Repositories](#) [Policies](#) [Restore Status](#)



You don't have any snapshots yet

Run a Snapshot Lifecycle Policy to create a snapshot. You can also create a snapshot using [the Elasticsearch API](#).

[+ Create a policy](#)

Si se ingresa a esta Interfaz se le presenta al usuario con lo siguiente:

Logistics

[Logistics docs](#)

Policy name

A unique identifier for this policy.

Name

Policy-ic4302

Snapshot name

The name for the snapshots. A unique identifier is automatically added to each name.

Snapshot name

snapshot-elastic-{now/d}>

Supports date math expressions. [Learn more.](#)

Repository

The repository where you want to store the snapshots.

Repository

BackupsElastic

Schedule

The frequency at which to take the snapshots.

Frequency

Every hour

Minute

At 30

[Create cron expression](#)

[Next >](#)

[Cancel](#)

Aquí se solicitan los siguientes datos del usuario:

- **Policy Name:** Aquí se le pone un nombre a la política, este valor es a gusto del usuario
- **Snapshot Name:** En esta sección se le debe de poner un nombre a los snapshots que se van a almacenar, la plataforma automáticamente les agrega un identificador único al final
- **Repository:** Aquí se debe poner el nombre del repositorio previamente creado, si no se ingresa un repositorio válido no se creará la política

- **Schedule:** En esta parte se debe definir cada cuanto tiempo se quieren hacer backups de la plataforma, se puede definir si se quiere cada ciertas horas, dias, mes, etc.

Una vez se han ingresado los datos, se pasa a la siguiente página:

1

2

3

4

Logistics

Snapshot settings

Snapshot retention

Review

Snapshot settings

[Snapshot docs](#)

Data streams and indices

To back up indices and data streams, manually select them or define index patterns to dynamically match them.

☒ All data streams and indices

Ignore unavailable indices

Ignores indices that are unavailable when taking the snapshot. Otherwise, the entire snapshot will fail.

☐ Ignore unavailable indices

Allow partial indices

Allows snapshots of indices with primary shards that are unavailable. Otherwise, the entire snapshot will fail.

☐ Allow partial indices

Include global state

Stores the global cluster state as part of the snapshot.

☒ Include global state

Include feature state

Includes the configuration, history, and other data stored in Elasticsearch by a feature such as Elasticsearch security.

☒ Include feature state from

All features

< Back

Next >

Cancel

En esta sección no es necesario cambiar nada, es cuestión de dejarlo con sus configuraciones por default, esto va a guardar todos los indices de la base automáticamente, sin embargo, si se quiere guardar solo un índice

Snapshot settings

[Snapshot docs](#)

Data streams and indices

To back up indices and data streams, manually select them or define index patterns to dynamically match them.

☐ All data streams and indices

Select indices and data streams

[Use index patterns](#)

☒ .logs-deprecation.elasticsearch-...

Data stream

☒ ilm-history-5

Data stream

☒ .kibana-event-log-8.6.1-000001

☒ indice-backup

Se puede apagar esta configuración, aqui se pueden seleccionar los indices que si se quieran guardar, si tienen un check a la izquierda van a ser parte del snapshot, sino no se incluyen.

Se pasa a la siguiente página:

Create policy

1

Logistics

2

Snapshot settings

3

Snapshot retention

4

Review

Snapshot retention (optional)

Snapshot docs

Expiration

The time to wait before deleting snapshots.

Delete after

3

days

Snapshots to retain

The minimum and maximum number of snapshots to store for the policy.

Minimum count

1

Maximum count

< Back

Next >

Cancel

Esta es la sección final para crear un policy, aquí se puede definir **Si se quieren borrar los backups después de cierto tiempo, y si se quieren definir un mínimo a mantener**. A través de esta sección se puede configurar los puntos extra de la tarea, indicando la cantidad de días tras los que se quiere borrar los backups.

Finalmente, le damos next y se va a crear el policy en esta sección:

Review policy

Summary

Request

Logistics

Policy name

Policy-ic4302

Repository

BackupsElastic

Snapshot settings

Data streams and indices

All indices

Include global state

Yes

Allow partial indices

No

Snapshot retention

Delete after

3d

Min count

1

Snapshot name

snapshot-elastic-{now/d}>

Schedule

0 30 * * * ?

Ignore unavailable indices

No

Include feature state

All features

< Back

Create policy

Cancel

Confirmamos que los datos están correctos y se le da crear,como resultado, si se hizo bien debe de aparecer lo siguiente:

×

SnapshotsPrueba-Elastic

[Summary](#) [History](#)

Snapshots taken

Failures

Deleted

Deletion failures

0

0

0

0

General

Version	Last modified
1	Oct 21, 2023 12:00 PM CST
Snapshot name	Repository
<snapshot-prueba-elastic>	BackupsElastic
Schedule	Next snapshot
0 0 * * * ?	Oct 21, 2023 1:00 PM CST
Data streams and indices	Ignore unavailable indices
indice-backup	No
Include global state	Include feature state
Yes	All features
Allow partial shards	
No	

Retention

Delete after

×

[Close](#)

Manage policy 

En este ejemplo solo se esta haciendo backup del index llamado: **indice-backup**

Despues de crear el policy y repositorio se van a crear backups automáticamente cada cierto tiempo dependiendo de lo definido al crear el policy.

[Snapshots](#) [Repositories](#) [Policies](#) [Restore Status](#)

Search...						Repository ▾	Reload
☐ Snapshot	Repository	Indices	Shards	Failed shards	Date created ↓	Duration	Actions
☐ snapshot-elastic-2023.10.20-ppt2fdq7sheb11fiwdpdwg	BackupsElastic	3	11	0	Oct 19, 2023 8:29 PM CST	80s	📄 🗑️

Rows per page: 20 ▾

< [1](#) >

Verificar si se subieron al AWS Bucket los backups

```

bash-4.2# aws s3 ls s3://tec-ic4302-02-2023/2021022613/elastic/
      PRE indices/
      PRE tests-029ZYCoAQPkwCAGb96iPqQ/
      PRE tests-0Dr7orOPQquwAw9-vVhNCg/
      PRE tests-9rTJHyn5ScGhEs9GKhmdUw/
      PRE tests-0BI95eoTRbGjt7CAZ10x3Q/
      PRE tests-aJYsqkfhSZmcvEFJM1FyEg/
      PRE tests-pVeX0HAYS3e0GFW1zVWg2Q/
2023-10-20 02:30:05      3332 index-0
2023-10-20 03:07:38       154 index-1
2023-10-20 03:28:53      3609 index-2
2023-10-20 04:28:48      4885 index-3
2023-10-20 05:28:46      6161 index-4
2023-10-20 05:28:47         8 index.latest
2023-10-20 04:28:47     34548 meta-B80vykD1RKYTBDYwPafdw.dat
2023-10-20 05:28:46     34549 meta-FIwkKg7DQF2rsgp57j3Z3Q.dat
2023-10-20 03:28:52     34544 meta-WEqxs1mpT0ia7dXyCW3EtA.dat
2023-10-20 02:30:04     34313 meta-YPd0QeQGTWSzKvUJU5JaKg.dat
2023-10-20 04:28:47       738 snap-B80vykD1RKYTBDYwPafdw.dat
2023-10-20 05:28:46       735 snap-FIwkKg7DQF2rsgp57j3Z3Q.dat
2023-10-20 03:28:53       738 snap-WEqxs1mpT0ia7dXyCW3EtA.dat
2023-10-20 02:30:03       709 snap-YPd0QeQGTWSzKvUJU5JaKg.dat
bash-4.2#

```

Restore

Para probar la restauración de datos usando Kibana, se deben haber hecho los pasos anteriores y al menos se debe tener:

- Un índice con datos
- Un Snapshot del Índice

Una vez se tienen esto, la forma más sencilla de probar es desde la sección DEV TOOLS de la interfaz de kibana, borrar el índice y tratar de restaurar sus datos usando el snapshot.

Si hacemos un GET del índice queda claro que si existe y sus datos son los siguientes:

```
GET /indice-backup/_search
{
  "query": {
    "match": {
      "titulo": "Prueba Backups"
    }
  }
}
```

```
1- {
2   "took": 1,
3   "timed_out": false,
4   "shards": {
5     "total": 1,
6     "successful": 1,
7     "skipped": 0,
8     "failed": 0
9   },
10  "hits": {
11    "total": {
12      "value": 1,
13      "relation": "eq"
14    },
15    "max_score": 0.5753642,
16    "hits": [
17      {
18        "_index": "indice-backup",
19        "_id": "1lF6Tos8M5xh8Gc3X0gH",
20        "_score": 0.5753642,
21        "_source": {
22          "titulo": "Prueba Backups",
23          "contenido": "Este documento es una prueba de backups",
24          "integrantes": [
25            "Granados Retana Diego",
26            "Granados Retana Daniel",
27            "Mora Montes Diego",
28            "Gutierrez Conejo Eduardo",
29            "Cardona Quesada Jose Ricardo"
30          ]
31        }
32      }
33    ]
34  }
35 }
```

Ahora se procede a verificar que si existe un snapshot con ese index:

```
1
2 GET /indice-backup/_search
3 {
4   "query": {
5     "match": {
6       "titulo": "Prueba Backups"
7     }
8   }
9 }
10
```

```
11 GET /_snapshot/BackupsElastic/_all
```

```
1- {
2   "snapshots": [
3     {
4       "snapshot": "indice-backup_20231020",
5       "uuid": "ijst37f1RPeqwsVBQeJ1ew",
6       "repository": "BackupsElastic",
7       "version_id": "8060199",
8       "version": "8.6.1",
9       "indices": [
10        "indice-backup"
11      ],
12      "data_streams": [],
13      "include_global_state": false,
14      "state": "SUCCESS",
15      "start_time": "2023-10-20T23:33:05.740Z",
16      "start_time_in_millis": 1697844785740,
17      "end_time": "2023-10-20T23:33:08.342Z",
18      "end_time_in_millis": 1697844788342,
19      "duration_in_millis": 2602,
20      "failures": [],
21      "shards": {
22        "total": 1,
23        "failed": 0,
24        "successful": 1
25      },
26      "feature_states": []
27    }
28  ],
29  "total": 1,
30  "remaining": 0
31 }
```

Para esto se uso el comando: **GET /_snapshot/BackupsElastic/_all**

Una vez verificados ambos, se va a borrar el índice de la base de datos elasticSearch:

```
1
2 GET /indice-backup/_search
3 {
4   "query": {
5     "match": {
6       "titulo": "Prueba Backups"
7     }
8   }
9 }
10
11 GET /_snapshot/BackupsElastic/_all
12
13 DELETE /indice-backup
```

```
1- {
2   "acknowledged": true
3 }
```

Se utiliza el comando: **DELETE /indice-backup**

Revisando si todavía existen los datos usando GET, se obtiene:

```
1
2 GET /indice-backup/_search
3 {}
4 {
5   "query": {
6     "match": {
7       "titulo": "Prueba Backups"
8     }
9   }
10 }
11 GET /_snapshot/BackupsElastic/_all
12
```

```
1- {
2   "error": {
3     "root_cause": [
4       {
5         "type": "index_not_found_exception",
6         "reason": "no such index [indice-backup]",
7         "resource.type": "index_or_alias",
8         "resource.id": "indice-backup",
9         "index_uuid": "_na_",
10        "index": "indice-backup"
11      }
12    ],
```

Restauración

Una vez eliminado, hay dos maneras de restaurar el índice dentro de Kibana:

Comandos

```
POST /_snapshot/[Nombre Repo]/[Nombre Snapshot]/_restore { "indices": "[Indice a restaurar]",
"include_global_state": false }
```

```
POST /_snapshot/BackupsElastic/indice-backup_20231020/_restore { "indices": "indice-backup",  
"include_global_state": false }
```

Ahora si probamos el GET, deberían de volver a aparecer los datos del Index:

```

1 GET /indice-backup/_search
2 {
3 {
4 "query": {
5 "match": {
6 "titulo": "Prueba Backups"
7 }
8 }
9 }
10
11 GET /_snapshot/BackupsElastic/_all
12
13 DELETE /indice-backup
14
15
16 POST /_snapshot/BackupsElastic/indice-backup_20231020/_restore
17 {
18 "indices": "indice-backup",
19 "include_global_state": false
20 }

```

```

1 {
2 "took": 1,
3 "timed_out": false,
4 "_shards": {
5 "total": 1,
6 "successful": 1,
7 "skipped": 0,
8 "failed": 0
9 },
10 "hits": {
11 "total": {
12 "value": 1,
13 "relation": "eq"
14 },
15 "max_score": 0.5753642,
16 "hits": [
17 {
18 "_index": "indice-backup",
19 "_id": "ILF6TosBW5xh8Gc3X0gH",
20 "_score": 0.5753642,
21 "_source": {
22 "titulo": "Prueba Backups",
23 "contenido": "Este documento es una prueba de backups",
24 "integrantes": [
25 "Granados Retana Diego",
26 "Granados Retana Daniel",
27 "Mora Montes Diego",
28 "Gutierrez Conejo Eduardo",
29 "Cardona Quesada Jose Ricardo"
30 ]
31 }
32 }
33 ]
34 }
35 }

```

La segunda opción para restaurar un índice eliminado es a través de la Sección Snapshot and Restore dentro de la Interfaz de Kibana en la sección de Stack Management. Una vez aquí buscamos lo siguiente:

☐ Snapshot	Repository	Indices	Shards	Failed shards	Date created ↓	Duration	Actions
☐ snapshot-prueba-elastic-za4l15ceslan4z5ezsmkr	BackupsElastic	1	9	0	Oct 22, 2023 6:59 PM CST	4s	
☐ snapshot-prueba-elastic-rjkrj6ds2-c1oyq5qcvwa	BackupsElastic	1	9	0	Oct 22, 2023 5:59 PM CST	4s	
☐ snapshot-prueba-elastic-m4thordbri2qdkaxv2n8xg	BackupsElastic	1	9	0	Oct 22, 2023 4:59 PM CST	4s	
☐ snapshot-prueba-elastic-qzhcuudwrresd8aodqnbxw	BackupsElastic	1	9	0	Oct 22, 2023 3:59 PM CST	4s	
☐ snapshot-prueba-elastic-hfkgvti-s56khs1jhjts6w	BackupsElastic	1	9	0	Oct 22, 2023 2:59 PM CST	5s	
☐ snapshot-prueba-elastic-lpx9nubhtwskvmuzyn5bfa	BackupsElastic	1	9	0	Oct 22, 2023 1:59 PM CST	4s	
☐ snapshot-prueba-elastic-22r2t5-jsoipg4eoo4b3nw	BackupsElastic	1	9	0	Oct 22, 2023 12:59 PM CST	5s	
☐ snapshot-prueba-elastic-taegbq4sstwwgnoc6_ioag	BackupsElastic	1	9	0	Oct 22, 2023 11:59 AM CST	5s	
☐ snapshot-prueba-elastic-	BackupsElastic	1	9	0	Oct 22, 2023 11:08 AM CST	5s	

Aqui aparecerán todos los snapshots creados, ya sea que se crearon manualmente con comandos o a través de una política. Estos se encuentran en orden descendente del creado más reciente al más antiguo. Se recomienda usar snapshots más nuevos para regresar a un estado más reciente de la base de datos en caso de un fallo.

Se selecciona el snapshot que se quiera restaurar y se clickea para ver detalles de este. Una vez abierto podremos ver la opción de restaurar:

SnapshotsRepositoriesPoliciesRestore Status

Q Search...

☐ Snapshot	Repository	Indices	Shards	Failed shards
☐ snapshot-prueba-elastic-za4l15ceslan4z5ezsmkr	BackupsElastic	1	9	0
☐ snapshot-prueba-elastic-rjkrj6ds2-c1oyq5qcvwa	BackupsElastic	1	9	0
☐ snapshot-prueba-elastic-m4thordbri2qdkaxv2n8xg	BackupsElastic	1	9	0
☐ snapshot-prueba-elastic-qzhcuudwrresd8aodqnbxw	BackupsElastic	1	9	0
☐ snapshot-prueba-elastic-hfkgvti-s56khs1jhjts6w	BackupsElastic	1	9	0
☐ snapshot-prueba-elastic-lpx9nubhtwskvmuzyn5bfa	BackupsElastic	1	9	0
☐ snapshot-prueba-elastic-22r2t5-jsoipg4eoo4b3nw	BackupsElastic	1	9	0
☐ snapshot-prueba-elastic-taegbq4sstwwgnoc6_ioag	BackupsElastic	1	9	0
☐ snapshot-prueba-elastic-	BackupsElastic	1	9	0

snapshot-prueba-elastic-za4l15ceslan4z5ezsmkr

'BackupsElastic' repository

SummaryFailed indices (0)

Version / Version ID8.6.1 / 8060199

State✔ Snapshot complete

Includes global stateYes

Indices (1)index-backup

Data streams (0)

Start timeOct 22, 2023 6:59 PM CST

Created bySnapshotsPrueba-Elastic

UUIDf57JLuSEQiuHywMIqZZOCA

Duration4 seconds

Include feature state fromgeopikibanasecurity

End timeOct 22, 2023 7:00 PM CST

Close

Delete

Restore

Si seleccionamos esta opción se desplegará un menú interactivo de restauración donde se pueden seleccionar los índices del snapshot que se quieren restaurar, si se quieren mantener los mismos nombres para los índices o cambiarlos, etc. A continuación se presenta este menú:

Restore 'snapshot-prueba-elastic-za4l15ceslan4z5ezsmkrg'

1

2

3

LogisticsIndex settingsReview

Restore details

Snapshot and Restore docs

Data streams and indices

Creates new data streams and indices if they don't exist. Opens existing indices, including backing indices for a data stream, if they are closed and have the same number of shards as the snapshot index.

All data streams and indices

Rename data streams and indices

Renames data streams and indices on restore. Ensure that a matching index template exists for renamed data streams.

Rename data streams and indices

Partial restore

Allows restore of indices that don't have snapshots of all shards.

Partial restore

Restore global state

Restores the global cluster state as part of the snapshot.

Restore global state

Restore feature state

Restores the configuration, history, and other data stored in Elasticsearch by a feature such as Elasticsearch security.

Restore feature state from

Restore aliases

Restores index aliases along with their associated indices.

Restore aliases

> Next

Seleccionamos las configuraciones que se deseen aplicar al restore y se pasa a la siguiente página:

Restore 'snapshot-prueba-elastic-za4l15ceslan4z5ezsmkrg'

✓

✓

3

LogisticsIndex settingsReview

Review restore details

SummaryJSON

Logistics

Data streams and indices

All indices

Include feature state

No

Index settings

No index setting modifications

< Back

✓ Restore snapshot

Se le da restore snapshot y listo, los datos de la base junto con los indices y configuraciones en el snapshot se restauraron a la base.

Pruebas Realizadas

21 / 35

Para las pruebas realizadas en elasticSearch, se crearon dos índices:

```
1 PUT pokemon-names
2 {
3   "mappings": {
4     "properties": {
5       "name": { "type": "keyword" }
6     }
7   }
8 }
9
10 PUT pokemon-regions
11 {
12   "mappings": {
13     "properties": {
14       "region": { "type": "keyword" }
15     }
16   }
17 }
18 }
```

```
1 {
2   "acknowledged": true,
3   "shards_acknowledged": true,
4   "index": "pokemon-regions"
5 }
```

A continuación se presentan los comandos para crear los índices e insertar datos en cada uno:

pokemon-names

```
PUT pokemon-names { "mappings": { "properties": { "name": { "type": "keyword" }
} } }
```

>>>> Datos:

```
POST pokemon-names/_doc { "name": "Pikachu" }
```

```
POST pokemon-names/_doc
{ "name": "Charmander" }
```

Consultando al índice se obtienen los datos:

```

1 {
2   "took": 1,
3   "timed_out": false,
4   "_shards": {
5     "total": 1,
6     "successful": 1,
7     "skipped": 0,
8     "failed": 0
9   },
10  "hits": {
11    "total": {
12      "value": 2,
13      "relation": "eq"
14    },
15    "max_score": 1,
16    "hits": [
17      {
18        "_index": "pokemon-names",
19        "_id": "4GZcWosB2vwyjbG-KKCs",
20        "_score": 1,
21        "_source": {
22          "name": "Pikachu"
23        }
24      },
25      {
26        "_index": "pokemon-names",
27        "_id": "4WZcWosB2vwyjbG-MqAX",
28        "_score": 1,
29        "_source": {
30          "name": "Charmander"
31        }
32      }
33    ]
34  }
35 }

```

El comando para consultar el índice es: **GET pokemon-names/_search**

pokemon-regions

PUT pokemon-regions { "mappings": { "properties": { "region": { "type": "keyword" } } } }

>>>> Datos:

POST pokemon-regions/_doc { "region": "Kanto" }

POST pokemon-regions/_doc { "region": "Johto" }

Consultando al indice se obtienen los datos:

```
1 {
2   "took": 1,
3   "timed_out": false,
4   "_shards": {
5     "total": 1,
6     "successful": 1,
7     "skipped": 0,
8     "failed": 0
9   },
10  "hits": {
11    "total": {
12      "value": 2,
13      "relation": "eq"
14    },
15    "max_score": 1,
16    "hits": [
17      {
18        "_index": "pokemon-regions",
19        "_id": "4mZcWosB2vwyjbG-gqCm",
20        "_score": 1,
21        "_source": {
22          "region": "Kanto"
23        }
24      },
25      {
26        "_index": "pokemon-regions",
27        "_id": "42ZcWosB2vwyjbG-iqCD",
28        "_score": 1,
29        "_source": {
30          "region": "Johto"
31        }
32      }
33    ]
34  }
35 }
```

En base a estos índices se estableció una política que guarde los datos asociados a estos dos índices cada hora. La política es:



snapshots-pokemon

Summary

History

Snapshots taken	Failures	Deleted	Deletion failures
0	0	0	0

General

Version	Last modified
1	Oct 22, 2023 8:47 PM CST
Snapshot name	Repository
pokemon-snapshots-	BackupsElastic
Schedule	Next snapshot
0 55 * * * ?	Oct 22, 2023 8:55 PM CST
Data streams and indices	Ignore unavailable indices
pokemon-names pokemon-regions	No
Include global state	Include feature state
Yes	All features
Allow partial shards	
No	

Ahora se van a borrar los dos índices y tratar de restaurarlos a través de hacer 2 experimentos:

Se borran los índices:

DELETE pokemon-names

DELETE pokemon-regions

Probando obtener los datos:

```
1
2
3
4
5
6
7 DELETE pokemon-names
8
9 DELETE pokemon-regions
10
11 GET pokemon-regions/_search
12
13 GET pokemon-names/_search
14
15
16
17
18
19
20
21
22
```

```
1 {
2   "error": {
3     "root_cause": [
4       {
5         "type": "index_not_found_exception",
6         "reason": "no such index [pokemon-regions]",
7         "resource.type": "index_or_alias",
8         "resource.id": "pokemon-regions",
9         "index_uuid": "_na_",
10        "index": "pokemon-regions"
11      }
12    ],
13    "type": "index_not_found_exception",
14    "reason": "no such index [pokemon-regions]",
15    "resource.type": "index_or_alias",
16    "resource.id": "pokemon-regions",
17    "index_uuid": "_na_",
18    "index": "pokemon-regions"
19  },
20  "status": 404
21 }
```

Experimento #1 (Restaurar Un solo índice y el otro no)

Restore 'pokemon-snapshots--fdbesic8rcuu8h1nedh0tw'

1

Logistics

2

Index settings

3

Review

Restore details

Snapshot and Restore docs

Data streams and indices

Creates new data streams and indices if they don't exist. Opens existing indices, including backing indices for a data stream, if they are closed and have the same number of shards as the snapshot index.

×

All data streams and indices

Select data streams and indices

Use index patterns

Filter options

pokemon-names

✓ pokemon-regions

1 index and 0 data streams will be restored. [Deselect all](#)

Rename data streams and indices

Renames data streams and indices on restore. Ensure that a matching index template exists for renamed data streams.

✓

Rename data streams and indices

Capture pattern

Replacement pattern

pokemon-regions

pokemon-regions12

Use regular expressions

Se va a restaurar solamente el índice **pokemon-region** con el nombre "**pokemon-regions12**"

Haciendo el restore, se consultan ambos índices:

Consultando el índice **pokemon-regions12**

26 / 35

```

1 {
2   "took": 1,
3   "timed_out": false,
4   "_shards": {
5     "total": 1,
6     "successful": 1,
7     "skipped": 0,
8     "failed": 0
9   },
10  "hits": {
11    "total": {
12      "value": 2,
13      "relation": "eq"
14    },
15    "max_score": 1,
16    "hits": [
17      {
18        "_index": "pokemon-regions12",
19        "_id": "4mZcWosB2vwyjbG-gqCm",
20        "_score": 1,
21        "_source": {
22          "region": "Kanto"
23        }
24      },
25      {
26        "_index": "pokemon-regions12",
27        "_id": "42ZcWosB2vwyjbG-iqCD",
28        "_score": 1,
29        "_source": {
30          "region": "Johto"
31        }
32      }
33    ]
34  }
35 }

```

Consultando el índice pokemon-regions

```

1 {
2   "error": {
3     "root_cause": [
4       {
5         "type": "index_not_found_exception",
6         "reason": "no such index [pokemon-regions]",
7         "resource.type": "index_or_alias",
8         "resource.id": "pokemon-regions",
9         "index_uuid": "_na_",
10        "index": "pokemon-regions"
11      }
12    ],
13    "type": "index_not_found_exception",
14    "reason": "no such index [pokemon-regions]",
15    "resource.type": "index_or_alias",
16    "resource.id": "pokemon-regions",
17    "index_uuid": "_na_",
18    "index": "pokemon-regions"
19  },
20  "status": 404
21 }

```

Como se puede ver ahora se restauró el índice pokemon-regions con el nombre **pokemon-regions12**.

También se puede observar que al consultar el índice **pokemon-names**, ya no existe:

```
1 {
2   "error": {
3     "root_cause": [
4       {
5         "type": "index_not_found_exception",
6         "reason": "no such index [pokemon-names]",
7         "resource.type": "index_or_alias",
8         "resource.id": "pokemon-names",
9         "index_uuid": "_na_",
10        "index": "pokemon-names"
11      }
12    ],
13    "type": "index_not_found_exception",
14    "reason": "no such index [pokemon-names]",
15    "resource.type": "index_or_alias",
16    "resource.id": "pokemon-names",
17    "index_uuid": "_na_",
18    "index": "pokemon-names"
19  },
20  "status": 404
21 }
```

Experimento #2 (Restaurar ambos índices después de Experimento1)

Para este experimento se hace un restore normal del snapshot completo:

Restore 'pokemon-snapshots--fdbesic8rcuu8h1nedh0tw'

1

2

3

Logistics

Index settings

Review

Restore details

[Snapshot and Restore docs](#)

Data streams and indices

Creates new data streams and indices if they don't exist. Opens existing indices, including backing indices for a data stream, if they are closed and have the same number of shards as the snapshot index.

☒

All data streams and indices

Rename data streams and indices

Renames data streams and indices on restore. Ensure that a matching index template exists for renamed data streams.

☐

Rename data streams and indices

Partial restore

Allows restore of indices that don't have snapshots of all shards.

☐

Partial restore

Restore global state

Restores the global cluster state as part of the snapshot.

☐

Restore global state

Restore feature state

Restores the configuration, history, and other data stored in Elasticsearch by a feature such as Elasticsearch security.

☐

Restore feature state from

Restore aliases

Restores index aliases along with their associated indices.

☒

Restore aliases

> Next

Después de hacer esto, se puede observar en la sección de Restored dentro de la interfaz de Kibana lo siguiente:

28 / 35

Snapshot and Restore

Use repositories to store and recover backups of your Elasticsearch indices and clusters.

Snapshots

Repositories

Policies

Restore Status

Refresh data every 30 seconds

Index	Status	Last activity	Shards completed	Shards in progress
pokemon-regions	Complete	Oct 22, 2023 9:47 PM CST	1	0
pokemon-names	Complete	Oct 22, 2023 9:47 PM CST	1	0
pokemon-regions12	Complete	Oct 22, 2023 9:32 PM CST	1	0

Rows per page: 20

Se muestran los archivos de los 3 índices restaurados, debido a que en el experimento 1 no se restauró el índice **Pokemon-regions** con el mismo nombre, se restauran 2 veces en vez de sobrescribir el índice restaurado previamente con la versión nueva del snapshot

```
1 {
2   "took": 1,
3   "timed_out": false,
4   "_shards": {
5     "total": 1,
6     "successful": 1,
7     "skipped": 0,
8     "failed": 0
9   },
10  "hits": {
11    "total": {
12      "value": 2,
13      "relation": "eq"
14    },
15    "max_score": 1,
16    "hits": [
17      {
18        "_index": "pokemon-regions",
19        "_id": "4mZcWosB2vwyjbG-gqCm",
20        "_score": 1,
21        "_source": {
22          "region": "Kanto"
23        }
24      },
25      {
26        "_index": "pokemon-regions",
27        "_id": "42ZcWosB2vwyjbG-iqCD",
28        "_score": 1,
29        "_source": {
30          "region": "Johto"
31        }
32      }
33    ]
34  }
35 }
```

Revisando **Pokemon-regions**

Pokemon-regions12:

```
1 {  
2   "took": 1,  
3   "timed_out": false,  
4   "_shards": {  
5     "total": 1,  
6     "successful": 1,  
7     "skipped": 0,  
8     "failed": 0  
9   },  
10  "hits": {  
11    "total": {  
12      "value": 2,  
13      "relation": "eq"  
14    },  
15    "max_score": 1,  
16    "hits": [  
17      {  
18        "_index": "pokemon-regions12",  
19        "_id": "4mZcWosB2vwyjbG-gqCm",  
20        "_score": 1,  
21        "_source": {  
22          "region": "Kanto"  
23        }  
24      },  
25      {  
26        "_index": "pokemon-regions12",  
27        "_id": "42ZcWosB2vwyjbG-iqCD",  
28        "_score": 1,  
29        "_source": {  
30          "region": "Johto"  
31        }  
32      }  
33    ]  
34  }  
35 }
```

Y si se trata de obtener **Pokemon-names**

```

1 {
2   "took": 0,
3   "timed_out": false,
4   "_shards": {
5     "total": 1,
6     "successful": 1,
7     "skipped": 0,
8     "failed": 0
9   },
10  "hits": {
11    "total": {
12      "value": 2,
13      "relation": "eq"
14    },
15    "max_score": 1,
16    "hits": [
17      {
18        "_index": "pokemon-names",
19        "_id": "4GZcWosB2vwyjbG-KKCs",
20        "_score": 1,
21        "_source": {
22          "name": "Pikachu"
23        }
24      },
25      {
26        "_index": "pokemon-names",
27        "_id": "4WZcWosB2vwyjbG-MqAX",
28        "_score": 1,
29        "_source": {
30          "name": "Charmander"
31        }
32      }
33    ]
34  }
35 }

```

Como se puede ver, cuando se restaura un índice desde un snapshot, se está creando un índice nuevo a partir de una copia de seguridad. Si se restaura un índice con el mismo nombre que ya existía previamente, Elasticsearch sobrescribirá el índice existente con la versión restaurada desde el snapshot.

Sin embargo, si se restaura el índice con un nombre diferente al que tenía originalmente, Elasticsearch creará un nuevo índice con ese nuevo nombre, en lugar de sobrescribir.

CouchDB

Para realizar la instalacion de CouchDB utilizamos la instalacion del [helm chart hecho por Apache](#).

Para realizar la instalación, se habilita en el `databases/values.yaml` y en `backups/values.yaml` en el folder de `helm` la opción de `enabled` para realizar la instalación de la base de datos deseada.

En el archivo de `databases/values.yaml`, se indica la versión de la imagen que se va a utilizar en este caso la 3.3.2 , también se define el usuario y la contraseña que se van a utilizar para poder acceder a la base de datos y por último se va a generar un UUID (Universal Unique Identifier) necesario para la creación de la base de datos, este lo creamos aleatoriamente mediante el uso del link (<https://www.uuidgenerator.net/api/version4>).

```

couchdb:
  enabled: true
  clusterSize: 1

```

```
adminUsername: "admin"
adminPassword: "admin"
image:
  tag: 3.3.2
couchdbConfig:
  couchdb:
    uuid: $(shell curl -s https://www.uuidgenerator.net/api/version4 2>/dev/null
| tr -d -)
```

Si se desea realizar un backup, en el archivo de backups/values.yaml se pone `type: backup` en el campo de `couchdb.config.type`. Los backups de CouchDB se van a guardar en `s3://tec-ic4302-02-2023/2019073558/couchdb/`. El campo de `schedule` debe contener un horario válido para un [CronJob](#). En nuestra configuración, está configurado para correr dos veces al día, 1 vez a medio día y otra vez a media noche. El host va a tener la dirección de la base de datos de CouchDB dentro del cluster para que así pueda localizarlo y hacer el backup, la bd va a ser la base de datos ya definida de la cual se va a hacer el backup, el port va a ser el predefinido por el helm chart de CouchDB por el cual vamos a tener acceso al API. Y tenemos el user y password que van a ser los mismos que definimos previamente, por ultimo para el caso del restore se le indica un name, que va a ser el nombre del archivo en el cual se está guardando el backup que se quiera restaurar.

```
couchdb:
  enabled: true
  config:
    namespace: default
    host: databases-couchdb.default.svc.cluster.local
    db: "pokemon"
    user: "admin"
    password: "admin"
    port: "5984"
    bucketName: tec-ic4302-02-2023
    path: 2019073558/couchdb
    maxBackups: 3
    name: couchdb-202310230113.json
    schedule: "0 */12 * * *"
    diskSize: 1
    storageClass: hostpath
    provider: aws
    type: backup
```

Luego se puede correr el script de `install.sh` en helm para correr el backup.

Cuando se instala en modo "backup", se crean dos objetos principales de Kubernetes: un CronJob que se va a ejecutar dos veces al día cada 12 horas, y un Job que se ejecuta de una vez y realiza el respaldo a la base de datos. Estos jobs van a ejecutar el script `TC2/helm/backups/scripts/couchdbB.sh`. Debido a que CouchDB no tiene definido ningún método para realizar los backups, hacemos uso del [API de CouchDB](#) para así extraer los documentos que contenga la base de datos y poder crear un respaldo de estos. Para realizar el backup utilizamos el siguiente metodo:


```
URL="http://$COUCHDB_USER:$COUCHDB_PSW@$COUCHDB_HOST:$COUCHDB_PORT/$COUCHDB_DB/_all_docs?include_docs=true"

curl -X GET "$URL" -H "Accept: application/json" | jq '{ docs: [.rows[] | .doc] }'
> /couchdump/$DATE/couchdb-$DATE.json
```

En donde de primero definimos el path que vamos a utilizar para hacer la llamada al API, en este caso utilizamos unas variables de entorno definidas con los datos del values.yaml, el endpoint que utilizamos es **db/_all_docs?include_docs=true**, el cual nos va a retornar todos los documentos de la base de datos que indiquemos para así poder almacenarlos en un archivo .json, ya teniendo el path para el llamado utilizamos el metodo **curl** para hacer el llamado, en este indicamos que es un GET, y le definimos el url, e indicamos el tipo de dato que esperamos de vuelta, al mismo tiempo estamos utilizando el método **jq** para así cambiarle el formato al archivo de respuesta para así tenerlo en un formato que luego pueda ser utilizado para el recovery, y por ultimo le indicamos el path en el cual queremos que se almacene y el nombre del archivo que queremos que tenga.

Finalmente, se utiliza la siguiente instrucción para subir el backup al bucket S3 de aws:

```
aws s3 cp /couchdump/$DATE s3://$BUCKET_NAME/$BACKUP_PATH/ --recursive
```

Este script crea backups dentro del S3 dentro de un archivo con la fecha de cuando se realizó el backup.

En el caso del restore, es bastante similar al backup pero tiene ciertas diferencias, entre estas esta que se utiliza el endpoint del API **_bulk_docs** al cual se le da el archivo .json extraído del path y con el nombre indicado e inserta en la base de datos todos los documentos dentro de este archivo.

Ejemplo de Backup y Restore:

Para tener más control, podemos instalar los helm charts por separado. Primero, hay que instalar bootstrap. Ubicándonos en el folder de helm, hacemos:

```
helm install bootstrap bootstrap
```

Luego, tras asegurarnos de que Couchdb está habilitado en el values.yaml de databases, hacemos:

```
helm install databases databases
```

Esto crea el pod de la base de datos.

Una vez creado hacemos un port forward dentro de lens del puerto de couchdb para así poder hacer uso del API en herramientas como Postman o Thunderclient.

Con esto ya podemos hacer la creación de la base de datos y la inserción de los archivos mediante llamados al API, para este ejemplo los llamados utilizados son los siguientes.

```
PUT http://127.0.0.1:50209/pokemon

PUT http://127.0.0.1:50209/pokemon/001
```

```
{"name": "Pikachu", "type": "Electric", "level": 25}
```

Ya con información en la base de datos, podemos instalar el helm chart de backups para que el Job realice el respaldo:

```
helm install backups backups
```

Podemos ir a Lens y la sección de Jobs. Aquí podemos abrir la consola del pod y ver el progreso del script. Si sale un error de que no encuentra el script, hay que hacer la instrucción en la consola de y reinstalar el helm chart de backups:

```
dos2unix backups/scripts/postgresqlBackup.sh
```

Una vez creado el backup volvemos a Postman y eliminamos nuestros datos en este caso el llamado utilizado es:

```
DELETE http://127.0.0.1:50209/pokemon/001?rev=1-a11edc90c0d2388866469ce2f5235047
```

Donde el rev es autogenerado por CouchDB por cada registro, podemos ver cual es el rev de nuestro registro con el llamado:

```
GET http://127.0.0.1:50209/pokemon/_all_docs?include_docs=true
```

Para realizar el restore, se escribe el nombre del backup generado, este lo recibimos cuando se ejecute el backup en el pod logs, este dato se agrega en el campo de name en el values.yaml de backups y cambiamos el type a restore. Tras hacer esto, podemos reinstalar el helm chart con:

```
helm uninstall backups
```

```
helm install backups backups
```

Cuando está en modo restore, no crea un CronJob, solo ejecuta un Job. Por lo tanto, podemos ver el progreso del Job en la pestaña de Jobs en Lens:

Al final, podemos saber que el Job terminó cuando la columna de Completions salga en 1/1.

Ahora mediante el mismo llamado en Postman:

```
GET http://127.0.0.1:50209/pokemon/_all_docs?include_docs=true
```

Y con el response se puede observar que el restore ha sido exitoso.

Conclusiones y Recomendaciones

Conclusiones

- El uso de almacenamiento en la nube con AWS S3 facilita la escalabilidad y accesibilidad de los backups, pero requiere tomar en cuenta consideraciones adicionales de seguridad a la hora de hacer el código y mantener su versionamiento.

- Si bien Elasticsearch requiere una configuración inicial más elaborada y el proceso para habilitar snapshots es más manual, una vez que se tienen los componentes configurados adecuadamente, la automatización de snapshots y restores resulta bastante sencilla y más fácil de entender desde una perspectiva de usuario

Recomendaciones

- Para implementar los backups desarrollados en este proyecto en un escenario real, se podrían implementar junto con herramientas de monitoreo como Prometheus y Grafana. Contar con dashboards personalizados en Grafana facilitaría identificar tendencias, bottlenecks y problemas en los procesos de backup.
- Para establecer backups en Elasticsearch, se recomienda hacer uso de la interfaz interactiva de Kibana para crear políticas de snapshots. Esto en lugar de ejecutar comandos manuales sobre la base ElasticSearch para hacer cada backup de forma individual, ya que provee una experiencia más amigable e intuitiva para los usuarios.
- En lugar de tener las credenciales de acceso a la nube y el bucket hardcoded en los scripts de backup, se recomienda almacenar estas credenciales como secrets de Kubernetes . De esta manera se centraliza la gestión de credenciales sensibles y se evita exponerlas en el código.
- Es crítico que el repositorio de código donde se almacena la tarea de backup se mantenga privado y con acceso restringido. De esta manera se evita que personas no deseadas puedan ver el código fuente y obtener las credenciales que dan acceso a la cuenta de AWS u otra plataforma en la nube.
- Para optimizar el uso de almacenamiento en AWS, se recomienda hacer uso de una política de retención de backups automatizada. En lugar de retener todos los backups por tiempo indefinido, usar una política conserve los backups más recientes y necesarios, y elimine versiones antiguas que ya no son relevantes.
- Es crítico validar periódicamente que al implementar backups, estos sean funcionales y permitan restaurar los datos correctamente. Por eso, al implementar componentes de backups se recomienda realizar pruebas regulares de restore a partir de los mismos para confirmar su correcto funcionamiento y capacidad de recuperación de información.
- En una tarea de este tipo, dividir el trabajo es importante, pero también es importante que cada persona del equipo entienda su tarea. Se recomienda mantener una buena comunicación durante el desarrollo del proyecto y nunca quedarse con dudas.
- A la hora de probar los backups generados, se recomienda hacer uso de un pod de depuración como el incluido en la solución de esta tarea. Dicho pod proporciona una terminal con conectividad directa al bucket de almacenamiento en AWS. Esto permite validar de forma efectiva que los respaldos se estén ejecutando correctamente y subiendo a la nube según lo esperado.